

ICS1513 - Computer Networks Laboratory

Experiment 7: Simulation of Networks using NS-3

Nitin H - 3122247001042

August 26 2026

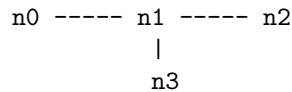
1 Simulation of a Simple Network using NS-3

Aim

To create and simulate a simple computer network with multiple nodes, point-to-point links, and traffic flows using NS-3, and to measure basic network performance (total bytes received, throughput of each flow, and average packet latency) without using FlowMonitor.

Question

Create a network consisting of four nodes (n0, n1, n2, and n3) connected using three point-to-point links, as shown below:



Configure the links with a data rate of 5 Mbps and a propagation delay of 2 ms. Assign appropriate IP addresses to all network interfaces and enable IP routing. Generate two UDP traffic flows:

- Flow 1: n0 → n2 at 1 Mbps
- Flow 2: n3 → n0 at 500 Kbps

Run the simulation for 10 seconds, calculate and display the total bytes received and throughput of each flow, and compare the performance of the two traffic flows.

Algorithm

1. Include the required NS-3 modules: `core-module`, `network-module`, `internet-module`, `point-to-point-module`, and `applications-module`.
2. Create a `NodeContainer` and instantiate 4 nodes (n0, n1, n2, n3) using `nodes.Create(4)`.
3. Configure a `PointToPointHelper` with a data rate of 5 Mbps and a propagation delay of 2 ms.
4. Install three point-to-point links using this helper: n0–n1, n1–n2, and n3–n2, producing three `NetDeviceContainers`.
5. Install the Internet protocol stack (IP, UDP/TCP, routing) on all four nodes using `InternetStackHelper`.
6. Assign IP addresses to each link's interfaces using `Ipv4AddressHelper`, using a separate /24 subnet per link (10.1.1.0, 10.1.2.0, 10.1.3.0).
7. Enable global IP routing across the topology using `Ipv4GlobalRoutingHelper::PopulateRoutingTables()`.
8. Configure Flow 1 (n0 → n2):

- (a) Install a `PacketSinkHelper` (UDP) on node `n2`, listening on port 5000, started at 1.0 s and stopped at 10.0 s.
 - (b) Install an `OnOffHelper` (UDP) on node `n0`, targeting `n2`'s address on port 5000, with a data rate of 1 Mbps and a packet size of 512 bytes, started at 2.0 s and stopped at 10.0 s.
9. Configure Flow 2 (`n3 → n0`) analogously: a sink on `n0` listening on port 6000, and a source on `n3` sending at 500 Kbps with 512-byte packets, started at 3.0 s and stopped at 10.0 s.
 10. Set the simulator to stop at 10.0 s using `Simulator::Stop()`, then run the simulation using `Simulator::Run()`.
 11. After the simulation completes, retrieve each flow's received byte count using `PacketSink::GetTotalRx()` on the respective sink applications, and print the results.
 12. Destroy the simulator using `Simulator::Destroy()`.

Code (Q71.cc)

```

1  #include "ns3/core-module.h"
2  #include "ns3/network-module.h"
3  #include "ns3/internet-module.h"
4  #include "ns3/point-to-point-module.h"
5  #include "ns3/applications-module.h"
6
7  /*core-module cl the simulation engine itself (the clock, event scheduler, logging)
8  network-module the basic building blocks: nodes, packets, network devices
9  internet-module IP addressing and routing (makes nodes act like real internet hosts)
10 point-to-point-module a specific type of link (a direct cable between exactly two nodes)
11 applications-module traffic generators/receivers (the things that actually send and receive data)*/
12
13 using namespace ns3;
14
15 NS_LOG_COMPONENT_DEFINE("SimpleNetwork");
16
17 int main()
18 {
19     NodeContainer nodes;
20     nodes.Create(4); //Create 4 Nodes
21
22     PointToPointHelper p2p;
23     p2p.SetDeviceAttribute("DataRate", StringValue("5Mbps")); //
24     p2p.SetChannelAttribute("Delay", StringValue("2ms"));
25
26     /*DataRate = 5 Mbps how fast the cable can push bits (bandwidth)
27     Delay = 2 ms how long it takes a signal to physically travel across the cable (propagation delay), like
28     the delay of light traveling down a fiber*/
29     NetDeviceContainer d01 = p2p.Install(nodes.Get(0), nodes.Get(1));
30     NetDeviceContainer d12 = p2p.Install(nodes.Get(1), nodes.Get(2));
31     NetDeviceContainer d32 = p2p.Install(nodes.Get(3), nodes.Get(2));
32
33     InternetStackHelper internet;
34     internet.Install(nodes);
35
36     //Assign IP addresses
37     Ipv4AddressHelper address;
38
39     address.SetBase("10.1.1.0", "255.255.255.0");
40     Ipv4InterfaceContainer i01 = address.Assign(d01);
41     address.SetBase("10.1.2.0", "255.255.255.0");
42     Ipv4InterfaceContainer i12 = address.Assign(d12);
43     address.SetBase("10.1.3.0", "255.255.255.0");
44     Ipv4InterfaceContainer i32 = address.Assign(d32);
45
46     //Enable Routing
47     Ipv4GlobalRoutingHelper::PopulateRoutingTables();
48     //Flow n0 - n2

```

```

48 uint16_t port1 = 5000;
49 PacketSinkHelper sink1("ns3::UdpSocketFactory", InetSocketAddress(Ipv4Address::GetAny(), port1));
50
51 ApplicationContainer sinkApp1 = sink1.Install(nodes.Get(2));
52 sinkApp1.Start(Seconds(1.0));
53 sinkApp1.Stop(Seconds(10.0));
54
55 OnOffHelper source1("ns3::UdpSocketFactory", InetSocketAddress(i12.GetAddress(1), port1));
56 source1.SetAttribute("DataRate", StringValue("1Mbps"));
57 source1.SetAttribute("PacketSize", UintegerValue(512));
58
59 ApplicationContainer sourceApp1 = source1.Install(nodes.Get(0));
60 sourceApp1.Start(Seconds(2.0));
61 sourceApp1.Stop(Seconds(10.0));
62
63 //Flow 2 n3 - n0: Same thing but at 500 Kbps
64 uint16_t port2 = 6000;
65 PacketSinkHelper sink2("ns3::UdpSocketFactory", InetSocketAddress(Ipv4Address::GetAny(), port2));
66
67 ApplicationContainer sinkApp2 = sink2.Install(nodes.Get(0));
68 sinkApp2.Start(Seconds(1.0));
69 sinkApp2.Stop(Seconds(10.0));
70
71 OnOffHelper source2("ns3::UdpSocketFactory", InetSocketAddress(i01.GetAddress(0), port2));
72 source2.SetAttribute("DataRate", StringValue("500Kbps"));
73 source2.SetAttribute("PacketSize", UintegerValue(512));
74
75 ApplicationContainer sourceApp2 = source2.Install(nodes.Get(3));
76 sourceApp2.Start(Seconds(3.0));
77 sourceApp2.Stop(Seconds(10.0));
78
79 //Run the Sim
80 Simulator::Stop(Seconds(10.0));
81 Simulator::Run();
82
83 // Display received packets
84 Ptr<PacketSink> sinkPtr1 = DynamicCast<PacketSink>(sinkApp1.Get(0));
85 Ptr<PacketSink> sinkPtr2 = DynamicCast<PacketSink>(sinkApp2.Get(0));
86
87 std::cout << "Flow 1 (n0->n2): "
88 << sinkPtr1->GetTotalRx()
89 << " bytes received" << std::endl;
90 std::cout << "Flow 2 (n3->n0): "
91 << sinkPtr2->GetTotalRx()
92 << " bytes received" << std::endl;
93 Simulator::Destroy();
94 return 0;
95 }

```

Output Screenshot

```

cseal@wtl-13:~/ICS1513-Computer-Networks-Labratry/Experiment 7/ns-allinone-3.43/ns-3.43$ ./ns3 run scratch/Q71
[0/2] Re-checking globbed directories...
ninja: no work to do.
Flow 1 (n0 -> n2): 499200 bytes received
Flow 2 (n3 -> n0): 187392 bytes received
cseal@wtl-13:~/ICS1513-Computer-Networks-Labratry/Experiment 7/ns-allinone-3.43/ns-3.43$

```

Figure 1: Simulation output showing total bytes received for Flow 1 ($n0 \rightarrow n2$) and Flow 2 ($n3 \rightarrow n0$)

Sample Input

None. All topology, link, and traffic-flow parameters are hardcoded in the program.

Sample Output

Flow 1 (n0 -> n2): 499200 bytes received

Flow 2 (n3 -> n0): 187392 bytes received

Throughput calculation (bytes received $\times$ 8 bits, divided by the flow's active duration):

- Flow 1 was active from 2.0 s to 10.0 s (8 s duration): throughput = $499200 \times 8 = 499200$ bps $\approx$ 0.499 Mbps.
- Flow 2 was active from 3.0 s to 10.0 s (7 s duration): throughput = $187392 \times 8 \approx 214163$ bps $\approx$ 0.214 Mbps.

Comparison: Flow 1 (configured at 1 Mbps) achieved roughly twice the received throughput of Flow 2 (configured at 500 Kbps), consistent with the source data rates configured for each `OnOffHelper`. Both flows achieved throughput below their configured source rate because the shared 5 Mbps/2 ms links, protocol/header overhead, and the `OnOffHelper`'s default on/off duty cycle reduce the effective, sustained data delivered within the simulation window.

Learning Outcomes

- Learnt to construct a multi-node, multi-link network topology in NS-3 using `NodeContainer` and `PointToPointHelper`.
- Learnt to install the Internet stack and assign subnetted IP addresses to point-to-point interfaces using `InternetStackHelper` and `Ipv4AddressHelper`.
- Learnt to enable global routing across multiple hops using `Ipv4GlobalRoutingHelper`.
- Learnt to generate and configure multiple simultaneous UDP traffic flows using `OnOffHelper` and `PacketSinkHelper`, and to measure total bytes received and throughput using `PacketSink::GetTotalRx()` without `FlowMonitor`.

2 UDP Traffic and Port-Based Application Delivery

Aim

To create a UDP communication between two nodes using NS-3, analyze the effect of point-to-point link latency on end-to-end throughput, and verify port-based delivery of multiple simultaneous UDP flows using packet capture and Wireshark.

Question

Create two nodes, Node1 and Node2, connected by a point-to-point link:

```
Node1 ----- Node2
      P2P Link
```

Install a `UdpClient` on Node1 and a `UdpServer` on Node2 on port 5000, and measure end-to-end throughput while varying the link latency (2 ms, 5 ms, 10 ms, 20 ms). Add a second `UdpClient/UdpServer` pair on port 6000, and repeat the throughput measurement with both UDP flows active. Enable PCAP tracing and use Wireshark to verify that packets destined for port 5000 and port 6000 are delivered to the correct server application. Record the observations in latency-vs-throughput tables.

Algorithm

1. Define a function `RunOnce()` that builds one topology, runs a single simulation for a given link delay, and returns the measured throughput(s) – this is called once per latency value tested.
2. Inside `RunOnce()`:
 - (a) Create a `NodeContainer` with 2 nodes (Node1, Node2).
 - (b) Configure a `PointToPointHelper` with a fixed link bandwidth (100 Mbps) and the given propagation delay (the loop variable).
 - (c) Install the point-to-point link between the two nodes, producing a `NetDeviceContainer`.
 - (d) Install the Internet stack on both nodes and assign IP addresses from a single /24 subnet (10.1.1.0).
 - (e) Configure Flow 1: install a `UdpServerHelper` on Node2 listening on port 5000; compute an appropriate packet interval from the desired data rate (5 Mbps) and packet size (1024 bytes); install a `UdpClientHelper` on Node1 targeting Node2's address and port 5000 with the computed interval, packet size, and a bounded maximum packet count.
 - (f) If a second flow is requested (`enableFlow2`), configure an identical second `UdpServerHelper/UdpClientHelper` pair on port 6000.
 - (g) Enable PCAP tracing for the link using `PointToPointHelper::EnablePcapAll()`, naming the trace file after the current delay and scenario.
 - (h) Run the simulation for a fixed duration (5 s of traffic plus setup/teardown margin) using `Simulator::Run()`.
 - (i) Compute each flow's throughput in Mbps from `UdpServer::GetReceived()`, the configured packet size, and the simulation duration.
 - (j) Destroy the simulator using `Simulator::Destroy()` so the next latency iteration starts from a clean state.
3. In `main()`, define a list of latency values to test: 2 ms, 5 ms, 10 ms, and 20 ms.
4. **Table 1 (single flow):** For each latency value, call `RunOnce()` with only Flow 1 enabled, and print the resulting latency-vs-throughput row.
5. **Table 2 (two flows):** For each latency value, call `RunOnce()` again with both Flow 1 and Flow 2 enabled, and print the resulting latency-vs-throughput-per-flow row.
6. After all iterations, report that per-latency/scenario PCAP files have been saved, to be opened in Wireshark and filtered with `udp.port == 5000` or `udp.port == 6000` to verify each flow reaches its correct destination port.

Code (Q72.cc)

```
1  /*
2  * =====
3  *  udp-two-ports-loop.cc
4  * =====
5  * Same experiment as before (2 nodes, 1 p2p link, UdpClient/UdpServer
6  * on ports 5000 and 6000) BUT this version loops through several
7  * latency values AUTOMATICALLY in one single run, and prints both
8  * result tables for you at the end. No need to re-run the command
9  * by hand for every latency.
10 *
11 * HOW TO RUN:
12 * ./ns3 run scratch/udp-two-ports-loop
13 *
14 * That's it one command gives you every row of both tables.
15 * =====
16 */
17
18 #include "ns3/core-module.h"
19 #include "ns3/network-module.h"
20 #include "ns3/internet-module.h"
21 #include "ns3/point-to-point-module.h"
22 #include "ns3/applications-module.h"
23 #include <vector>
24 #include <iomanip>
25
26 using namespace ns3;
27
28 NS_LOG_COMPONENT_DEFINE ("UdpTwoPortsLoop");
29
30 // -----
31 // This function builds the topology, runs ONE simulation for a
32 // given delay, and returns the throughput of flow1 (and flow2 if
33 // enabled). We call this function once per latency value in a loop.
34 // -----
35 void RunOnce (std::string delay,
36               bool enableFlow2,
37               double &throughput1Out,
38               double &throughput2Out)
39 {
40     std::string linkBandwidth = "100Mbps";
41     std::string dataRate1 = "5Mbps";
42     std::string dataRate2 = "5Mbps";
43     uint32_t packetSize = 1024;
44     double simTime = 5.0;
45
46     // STEP 1: topology
47     NodeContainer nodes;
48     nodes.Create (2);
49
50     PointToPointHelper p2p;
51     p2p.SetDeviceAttribute ("DataRate", StringValue (linkBandwidth));
52     p2p.SetChannelAttribute ("Delay", StringValue (delay)); // <- latency for this iteration
53
54     NetDeviceContainer devices = p2p.Install (nodes);
55
56     InternetStackHelper internet;
57     internet.Install (nodes);
58
59     Ipv4AddressHelper address;
60     address.SetBase ("10.1.1.0", "255.255.255.0");
61     Ipv4InterfaceContainer interfaces = address.Assign (devices);
62     Ipv4Address node2Address = interfaces.GetAddress (1);
63
64     // STEP 2: flow 1, port 5000
65     uint16_t port1 = 5000;
66     UdpServerHelper server1 (port1);
```

```

67 ApplicationContainer serverApps1 = server1.Install (nodes.Get (1));
68 serverApps1.Start (Seconds (0.0));
69 serverApps1.Stop (Seconds (simTime + 1.0));
70
71 DataRate rate1 (dataRate1);
72 double interval1 = (packetSize * 8.0) / rate1.GetBitRate ();
73 uint32_t maxPackets1 = (uint32_t) ((simTime / interval1) + 10);
74
75 UdpClientHelper client1 (node2Address, port1);
76 client1.SetAttribute ("MaxPackets", UIntegerValue (maxPackets1));
77 client1.SetAttribute ("Interval", TimeValue (Seconds (interval1)));
78 client1.SetAttribute ("PacketSize", UIntegerValue (packetSize));
79 ApplicationContainer clientApps1 = client1.Install (nodes.Get (0));
80 clientApps1.Start (Seconds (1.0));
81 clientApps1.Stop (Seconds (simTime + 1.0));
82
83 // STEP 3: flow 2, port 6000 (only if enabled)
84 ApplicationContainer serverApps2;
85 uint16_t port2 = 6000;
86 if (enableFlow2)
87 {
88     UdpServerHelper server2 (port2);
89     serverApps2 = server2.Install (nodes.Get (1));
90     serverApps2.Start (Seconds (0.0));
91     serverApps2.Stop (Seconds (simTime + 1.0));
92
93     DataRate rate2 (dataRate2);
94     double interval2 = (packetSize * 8.0) / rate2.GetBitRate ();
95     uint32_t maxPackets2 = (uint32_t) ((simTime / interval2) + 10);
96
97     UdpClientHelper client2 (node2Address, port2);
98     client2.SetAttribute ("MaxPackets", UIntegerValue (maxPackets2));
99     client2.SetAttribute ("Interval", TimeValue (Seconds (interval2)));
100    client2.SetAttribute ("PacketSize", UIntegerValue (packetSize));
101    ApplicationContainer clientApps2 = client2.Install (nodes.Get (0));
102    clientApps2.Start (Seconds (1.0));
103    clientApps2.Stop (Seconds (simTime + 1.0));
104 }
105
106 // Pcap file name includes the delay so each iteration gets its own file
107 std::string pcapName = "udp-two-ports-" + delay + (enableFlow2 ? "-both" : "-flow1only");
108 p2p.EnablePcapAll (pcapName);
109
110 Simulator::Stop (Seconds (simTime + 2.0));
111 Simulator::Run ();
112
113 Ptr<UdpServer> udpServer1 = DynamicCast<UdpServer> (serverApps1.Get (0));
114 throughput1Out = (udpServer1->GetReceived () * packetSize * 8.0) / (simTime * 1e6);
115
116 if (enableFlow2)
117 {
118     Ptr<UdpServer> udpServer2 = DynamicCast<UdpServer> (serverApps2.Get (0));
119     throughput2Out = (udpServer2->GetReceived () * packetSize * 8.0) / (simTime * 1e6);
120 }
121 else
122 {
123     throughput2Out = 0.0;
124 }
125
126 // IMPORTANT: reset the simulator so the next loop iteration starts fresh
127 Simulator::Destroy ();
128 }
129
130 int
131 main (int argc, char *argv[])
132 {
133     CommandLine cmd;
134     cmd.Parse (argc, argv);

```

```

135
136 // The latency values we want to test -- add/remove values here freely
137 std::vector<std::string> delays = {"2ms", "5ms", "10ms", "20ms"};
138
139 // -----
140 // TABLE 1: single flow (port 5000 only) -- Requirement #3
141 // -----
142 std::cout << "\n====TABLE_1:SingleUDPflow(port_5000_only)====\n";
143 std::cout << std::left << std::setw(12) << "Latency"
144         << std::setw(18) << "Throughput(Mbps)" << std::endl;
145
146 std::vector<double> table1Results;
147 for (const auto &delay : delays)
148 {
149     double t1 = 0.0, t2 = 0.0;
150     RunOnce (delay, false, t1, t2); // enableFlow2 = false
151     table1Results.push_back (t1);
152
153     std::cout << std::left << std::setw(12) << delay
154             << std::setw(18) << t1 << std::endl;
155 }
156
157 // -----
158 // TABLE 2: both flows active (ports 5000 and 6000) -- Requirement #7
159 // -----
160 std::cout << "\n====TABLE_2:TwoUDPflowsactive(ports_5000_&_6000)====\n";
161 std::cout << std::left << std::setw(12) << "Latency"
162         << std::setw(20) << "Flow1(Mbps)"
163         << std::setw(20) << "Flow2(Mbps)" << std::endl;
164
165 for (const auto &delay : delays)
166 {
167     double t1 = 0.0, t2 = 0.0;
168     RunOnce (delay, true, t1, t2); // enableFlow2 = true
169     std::cout << std::left << std::setw(12) << delay
170             << std::setw(20) << t1
171             << std::setw(20) << t2 << std::endl;
172 }
173
174 std::cout << "\nDone. Pcap files were saved per latency/scenario"
175         << "(e.g. udp-two-ports-5ms-both-0-0.pcap) -- open them in"
176         << "Wireshark and filter udp.port==5000 or udp.port==6000.\n";
177
178 return 0;
179 }
180
181 /*
182 * =====
183 * HOW TO USE
184 * =====
185 * 1) Copy this file into your ns-3 "scratch" folder:
186 * <ns3-folder>/scratch/udp-two-ports-loop.cc
187 *
188 * 2) Build:
189 * ./ns3 build
190 *
191 * 3) Run (single command, no flags needed):
192 * ./ns3 run scratch/udp-two-ports-loop
193 *
194 * It will print two tables straight to your terminal:
195 * TABLE 1 -> Latency vs Throughput (single flow)
196 * TABLE 2 -> Latency vs Flow1 Throughput vs Flow2 Throughput
197 *
198 * Just copy those numbers into your report.
199 *
200 * 4) Want to test different/more latencies? Edit this one line
201 * near the top of main():
202 * std::vector<std::string> delays = {"2ms", "5ms", "10ms", "20ms"};

```



```

203 * Add or remove values, e.g. {"1ms", "2ms", "5ms", "10ms", "20ms", "50ms"}.
204 *
205 * 5) Wireshark: every latency/scenario combo gets its own pcap file,
206 * named like:
207 * udp-two-ports-5ms-both-0-0.pcap (Node1 side)
208 * udp-two-ports-5ms-both-1-0.pcap (Node2 side)
209 * Filter with "udp.port == 5000" or "udp.port == 6000" to verify
210 * each flow reaches the correct server.
211 * =====
212 */

```

Output Screenshots

```

cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 7/ns-allinone-3.43/ns-3.43$ ./ns3 run Q72
[0/2] Re-checking globbed directories...
[2/2] Linking CXX executable ../build/scratch/ns3.43-Q72-default

===== TABLE 1: Single UDP flow (port 5000 only) =====
Latency      Throughput(Mbps)
2ms          4.99876
5ms          4.99548
10ms         4.99057
20ms         4.98074

===== TABLE 2: Two UDP flows active (ports 5000 & 6000) =====
Latency      Flow1(Mbps)      Flow2(Mbps)
2ms          4.99876        4.99876
5ms          4.99548        4.99548
10ms         4.99057        4.99057
20ms         4.98074        4.98074

cseal@wtl-13:~/ICS1513-Computer-Networks-Labratory/Experiment 7/ns-allinone-3.43/ns-3.43$ ls
AUTHORS          LICENSE          udp-two-ports-10ms-both-0-0.pcap  udp-two-ports-2ms-flow1only-1-0.pcap
bindings         LICESSES        udp-two-ports-10ms-both-1-0.pcap  udp-two-ports-5ms-both-0-0.pcap
build            ns3             udp-two-ports-10ms-flow1only-0-0.pcap  udp-two-ports-5ms-both-1-0.pcap
build-support    pyproject.toml  udp-two-ports-10ms-flow1only-1-0.pcap  udp-two-ports-5ms-flow1only-0-0.pcap
CHANGES.md      README.md       udp-two-ports-20ms-both-0-0.pcap  udp-two-ports-5ms-flow1only-1-0.pcap
cmake-cache      RELEASE_NOTES.md  udp-two-ports-20ms-both-1-0.pcap  utils
CMakeLists.txt   scratch         udp-two-ports-20ms-flow1only-0-0.pcap  utils.py
contrib          setup.cfg       udp-two-ports-20ms-flow1only-1-0.pcap  VERSION
CONTRIBUTING.md setup.py        udp-two-ports-2ms-both-0-0.pcap
doc              src            udp-two-ports-2ms-both-1-0.pcap
examples         test.py        udp-two-ports-2ms-flow1only-0-0.pcap

```

Figure 2: Simulation output: Table 1 (single UDP flow throughput vs. latency) and Table 2 (two UDP flows' throughput vs. latency), plus the generated per-scenario PCAP files

Sample Input

None. The link bandwidth, packet size, data rates, and the list of latency values to sweep are hardcoded in the program.

Sample Output

```

===== TABLE 1: Single UDP flow (port 5000 only) =====
Latency      Throughput(Mbps)
2ms          4.99876
5ms          4.99548
10ms         4.99057
20ms         4.98074

===== TABLE 2: Two UDP flows active (ports 5000 & 6000) =====
Latency      Flow1(Mbps)      Flow2(Mbps)

```

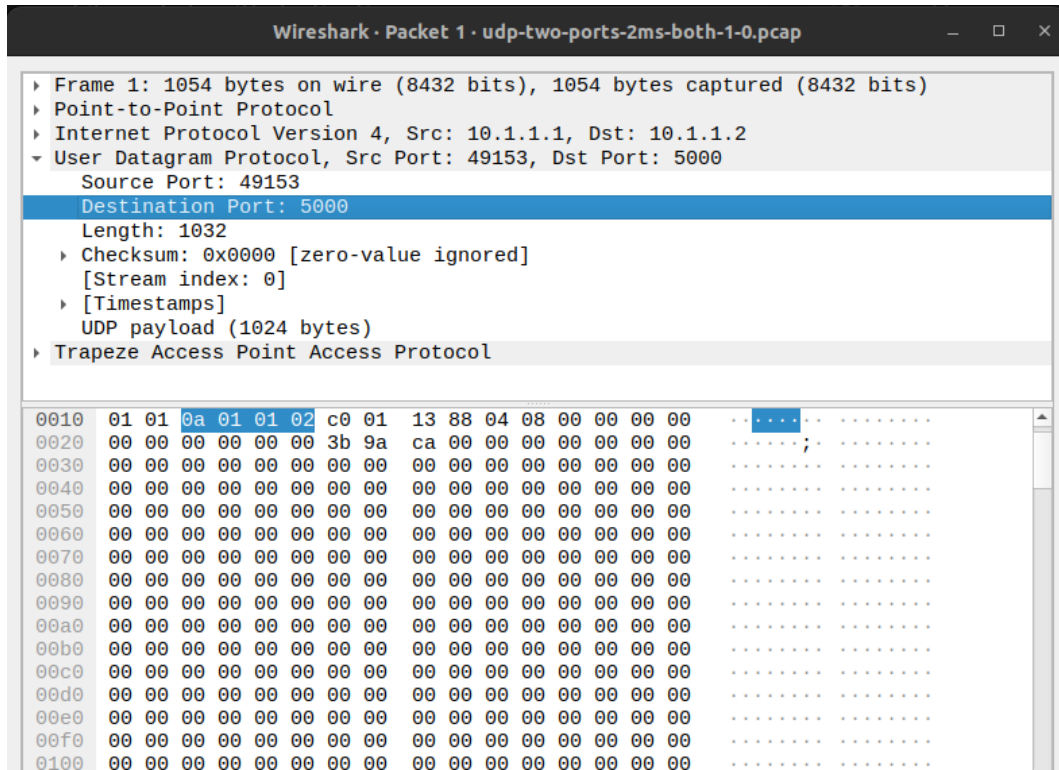


Figure 3: Wireshark capture of a packet from the 2ms/both-flows trace, showing UDP destination port 5000 (Flow 1)

2ms	4.99876	4.99876
5ms	4.99548	4.99548
10ms	4.99057	4.99057
20ms	4.98074	4.98074

Observation table:

Link Latency	Flow 1 Throughput	Flow 2 Throughput
2 ms	4.99876 Mbps	4.99876 Mbps
5 ms	4.99548 Mbps	4.99548 Mbps
10 ms	4.99057 Mbps	4.99057 Mbps
20 ms	4.98074 Mbps	4.98074 Mbps

The Wireshark capture (Fig. 3, Fig. 4) confirms that within the same PCAP trace, one packet is addressed to destination UDP port 5000 (Flow 1, delivered to Server 1) and another to destination UDP port 6000 (Flow 2, delivered to Server 2), verifying that the destination port number correctly distinguishes the two server applications sharing the same IP addresses.

Learning Outcomes

- Learnt to configure and run repeated NS-3 simulations programmatically over a range of link-latency values using a reusable simulation function.
- Learnt to install and configure `UdpClientHelper`/`UdpServerHelper` applications and compute throughput from `UdpServer::GetReceived()`.
- Understood how increasing point-to-point link latency causes a slight, measurable decrease in achieved UDP throughput even when the link bandwidth is unchanged.

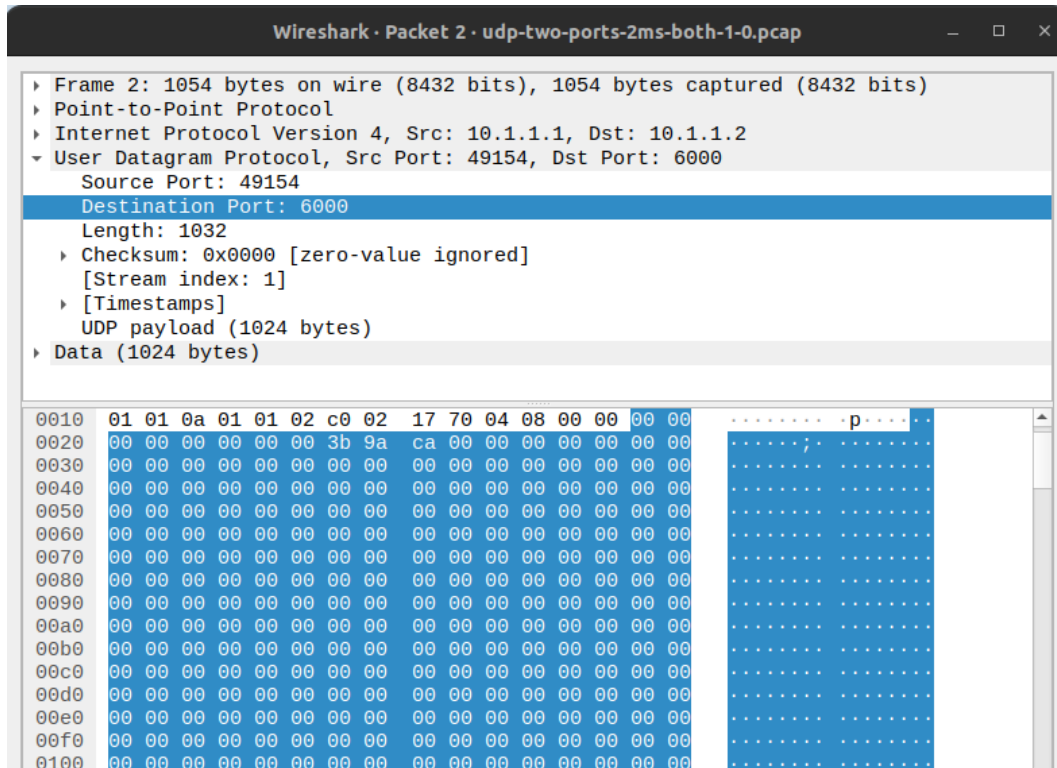


Figure 4: Wireshark capture of a packet from the same trace, showing UDP destination port 6000 (Flow 2), confirming port-based delivery to the correct server

- Learnt to distinguish multiple simultaneous UDP flows sharing the same pair of IP addresses using distinct destination port numbers, and to verify this behavior using PCAP tracing and Wireshark's `udp.port` filter.